# K-Nearest Neighbors (KNN) on Diabetes Dataset

**Members:** Felix Joseph Dinopol, Christian Jay Lucañas, John Kierve Gardonia

**Objective Overview**

This report details the application of the K-Nearest Neighbors (KNN) algorithm to predict diabetes outcomes based on clinical metrics. The process involves comprehensive data analysis, preprocessing, manual mathematical validation, algorithmic evaluation, and visual data correlation to determine the model's predictive capabilities. Furthermore, a bonus comparative analysis with Logistic Regression is included to contextualize KNN's performance.

**Part 1: Data Understanding**

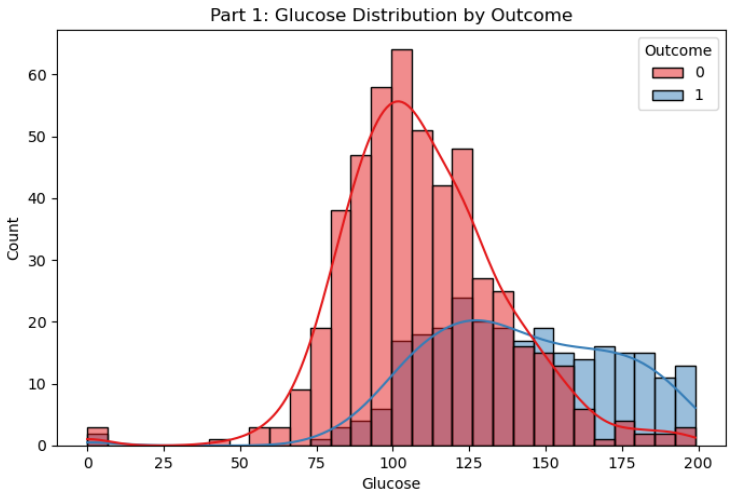
**1. Feature Descriptions** The dataset comprises 768 patient records with the following features:

- **Pregnancies:** The total number of times the patient has been pregnant.
- **Glucose:** The plasma glucose concentration measured after a 2-hour oral glucose tolerance test.
- **Blood Pressure:** Diastolic blood pressure reading (mm Hg).
- **SkinThickness:** Triceps skinfold thickness (in mm), utilized as an anatomical estimator of body fat.
- **Insulin:** 2-Hour serum insulin levels.
- **BMI:** Body Mass Index, a proportional metric of body fat based on a patient's weight and height.
- **Diabetes Pedigree Function:** A computed genetic score indicating the likelihood of diabetes based on family history.
- **Age:** The patient's age in years.
- **Outcome:** The target variable where 1 designates a diabetic patient and 0 designates a non-diabetic patient.

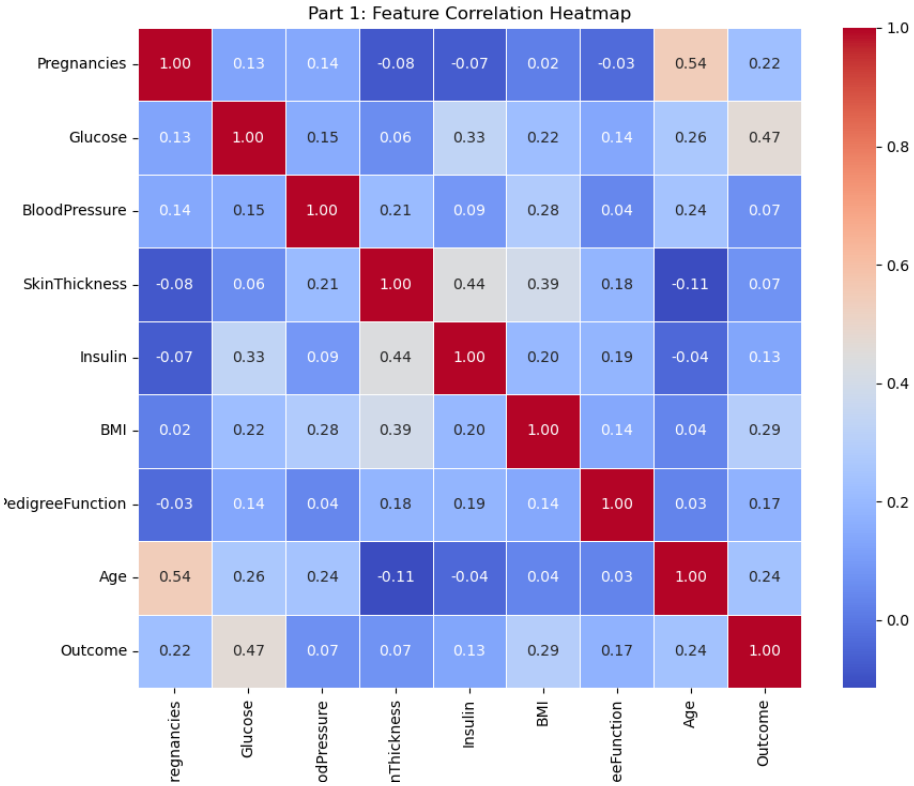
2. Feature Analysis & Visual Proof

- **Predictive Importance:** From a clinical perspective, Glucose is the strongest indicator for diagnosing diabetes. BMI, Age, and the Diabetes Pedigree Function also serve as highly critical predictive features. To mathematically prove this, a correlation heatmap was generated. Glucose demonstrates the highest positive correlation coefficient with the Outcome variable.
- **Problematic Data:** Upon initial inspection, several biological features (Glucose, Blood Pressure, SkinThickness, Insulin, and BMI) contained values of 0. Because it is biologically

impossible for a living human to have zero blood pressure or a BMI of zero, these values are clearly unrecorded or missing data masked as zeroes.




*Part 1: Glucose Distribution by Outcome)*




*Part 1: Feature Correlation Heatmap*

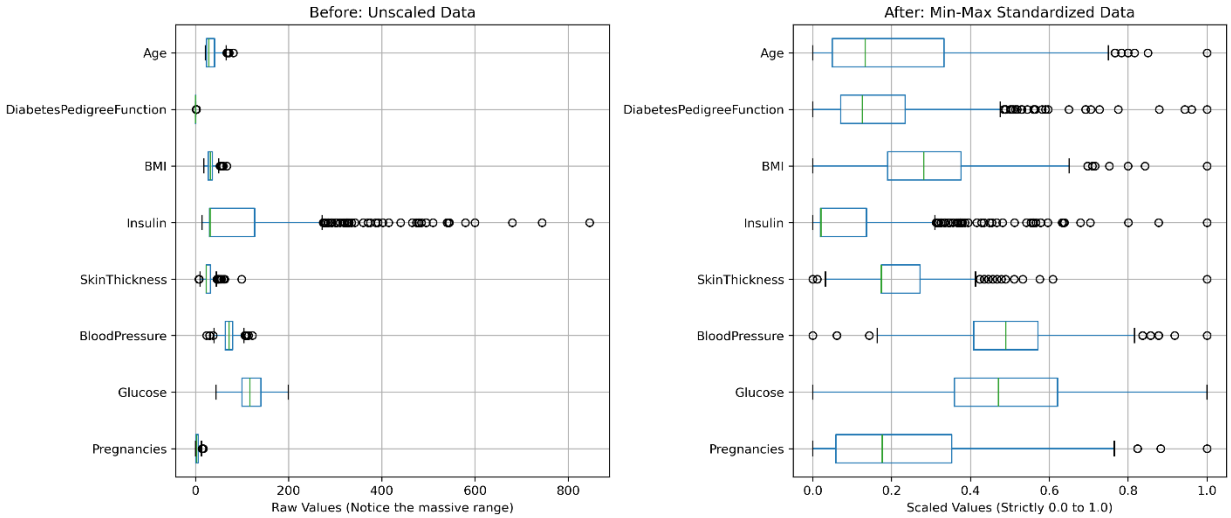
**Part 2: Data Preprocessing**

**1. Handling Missing Data.** Simply removing rows containing a zero in columns like Insulin or SkinThickness would eliminate a massive portion of the dataset, resulting in unacceptable information loss. Instead, Median Imputation was applied. All zeroes in the biological measurement columns were temporarily converted to NaN and then filled using the median value of their respective columns. The median is mathematically superior to the mean here because it is robust against extreme clinical outliers.

**2. Feature Scaling.** KNN is a distance-based algorithm. If left unscaled, features with large numerical ranges (e.g., Glucose) would geometrically overpower features with smaller ranges. To ensure all features contribute equally, Min-Max Normalization was strictly applied, transforming all values to a standardized scale between 0.0 and 1.0.

**3. Before & After Preprocessing Summary Table.** The following table and subsequent boxplot graph demonstrate the successful normalization of the problematic features:

| Feature | Original Median (With Zeros) | Corrected Median (Zeros Removed) | Scaled Range (Min-Max) |
|---|---|---|---|
| Glucose | 117.0 | 117.0 | 0.0 to 1.0 |
| BloodPressure | 72.0 | 72.0 | 0.0 to 1.0 |
| SkinThickness | 23.0 | 29.0 | 0.0 to 1.0 |
| Insulin | 30.5 | 125.0 | 0.0 to 1.0 |
| BMI | 32.0 | 32.3 | 0.0 to 1.0 |




*Part 2: Normalization Progress: Unscaled to Standardized*

**Part 3: KNN Implementation & Manual Computation**

The dataset was split using an 80% Training / 20% Testing ratio. To validate the algorithm's mechanics, a manual computation was performed on a single test instance against the first ten instances of the training set.

The standardized feature vector order is: [Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age]. Target Test Instance (Scaled): x = [0.353, 0.348, 0.347, 0.283, 0.212, 0.323, 0.150, 0.367]

**Distance Computation to 10 Training Samples (Euclidean Distance):** Using the formula:

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Step-by-step example for Training Sample 1:

$$d = \sqrt{(0.353 - 0.118)^2 + (0.348 - 0.258)^2 + \cdots + (0.367 - 0.0)^2} = \sqrt{0.2305} = 0.4801$$

**Manual Computation Summary Table:**

| Training Sample | Exact Mathematical Expansion: $\sqrt{[\ \Sigma(x\ -\ y)^2\ ]}$ | Final Distance (d) | Actual Outcome |
|---|---|---|---|
| **Training Sample 1** | $\sqrt{[\ (0.353-0.118)^2\ +\ (0.348-0.258)^2\ +\ (0.347-0.490)^2\ +\ (0.283-0.239)^2\ +\ (0.212-0.133)^2\ +\ (0.323-0.288)^2\ +\ (0.150-0.096)^2\ +\ (0.367-0.000)^2\ ]}\ =\ \sqrt{0.2305}$ | 0.4801 | 0 (Healthy) |
| **Training Sample 2** | $\sqrt{[\ (0.353-0.529)^2\ +\ (0.348-0.439)^2\ +\ (0.347-0.592)^2\ +\ (0.283-0.185)^2\ +\ (0.212-0.133)^2\ +\ (0.323-0.204)^2\ +\ (0.150-0.514)^2\ +\ (0.367-0.483)^2\ ]}\ =\ \sqrt{0.2750}$ | 0.5244 | 1 (Diabetic) |
| **Training Sample 3** | $\sqrt{[\ (0.353-0.059)^2\ +\ (0.348-0.613)^2\ +\ (0.347-0.224)^2\ +\ (0.283-0.130)^2\ +\ (0.212-0.083)^2\ +\ (0.323-0.215)^2\ +\ (0.150-0.246)^2\ +\ (0.367-0.017)^2\ ]}\ =\ \sqrt{0.3545}$ | 0.5954 | 0 (Healthy) |
| **Training Sample 4** | $\sqrt{[\ (0.353-0.000)^2\ +\ (0.348-0.755)^2\ +\ (0.347-0.265)^2\ +\ (0.283-0.239)^2\ +\ (0.212-0.133)^2\ +\ (0.323-0.076)^2\ +\ (0.150-0.075)^2\ +\ (0.367-0.733)^2\ ]}\ =\ \sqrt{0.5058}$ | 0.7112 | 0 (Healthy) |
| **Training Sample 5** | $\sqrt{[\ (0.353-0.353)^2\ +\ (0.348-0.581)^2\ +\ (0.347-0.571)^2\ +\ (0.283-0.326)^2\ +\ (0.212-0.428)^2\ +\ (0.323-0.573)^2\ +\ (0.150-0.068)^2\ +\ (0.367-0.417)^2\ ]}\ =\ \sqrt{0.2245}$ | 0.4738 | 1 (Diabetic) |

| Training Sample | Exact Mathematical Expansion: $\sqrt{[\,\Sigma(x\,-\,y)^2\,]}$ | Final Distance (d) | Actual Outcome |
|---|---|---|---|
| **Training Sample 6** | $\sqrt{[\,(0.353-0.059)^2\,+\,(0.348-0.555)^2\,+\,(0.347-0.469)^2\,+\,(0.283-0.065)^2+\,(0.212-0.109)^2\,+\,(0.323-0.157)^2\,+\,(0.150-0.168)^2+\,(0.367-0.017)^2\,]\,=\,\sqrt{0.3521}}$ | 0.5934 | 0 (Healthy) |
| **Training Sample 7** | $\sqrt{[\,(0.353-0.235)^2\,+\,(0.348-0.568)^2\,+\,(0.347-0.490)^2\,+\,(0.283-0.239)^2+\,(0.212-0.133)^2\,+\,(0.323-0.301)^2\,+\,(0.150-0.096)^2+\,(0.367-0.033)^2\,]\,=\,\sqrt{0.2049}}$ | 0.4527 | 1 (Diabetic) |
| **Training Sample 8** | $\sqrt{[\,(0.353-0.588)^2\,+\,(0.348-0.755)^2\,+\,(0.347-0.449)^2\,+\,(0.283-0.174)^2+\,(0.212-0.142)^2\,+\,(0.323-0.149)^2\,+\,(0.150-0.106)^2+\,(0.367-0.433)^2\,]\,=\,\sqrt{0.2843}}$ | 0.5332 | 1 (Diabetic) |
| **Training Sample 9** | $\sqrt{[\,(0.353-0.059)^2\,+\,(0.348-0.413)^2\,+\,(0.347-0.367)^2\,+\,(0.283-0.424)^2+\,(0.212-0.197)^2\,+\,(0.323-0.354)^2\,+\,(0.150-0.144)^2+\,(0.367-0.050)^2\,]\,=\,\sqrt{0.2125}}$ | 0.4610 | 0 (Healthy) |
| **Training Sample 10** | $\sqrt{[\,(0.353-0.059)^2\,+\,(0.348-0.232)^2\,+\,(0.347-0.316)^2\,+\,(0.283-0.239)^2+\,(0.212-0.133)^2\,+\,(0.323-0.018)^2\,+\,(0.150-0.077)^2+\,(0.367-0.000)^2\,]\,=\,\sqrt{0.3416}}$ | 0.5845 | 0 (Healthy) |

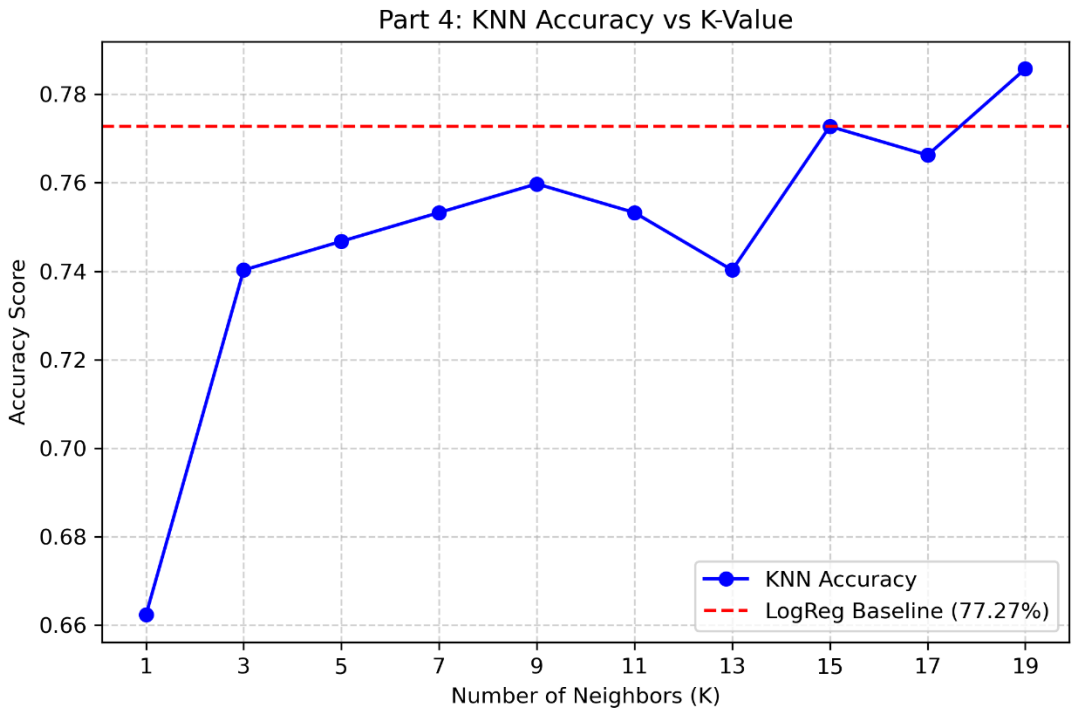
**Nearest Neighbors Classification (for K=3):** Sorting the distances from lowest to highest, the 3 nearest neighbors are Training Sample 7 (d = 0.4527, Class 1), Training Sample 9 (d = 0.4610, Class 0), and Training Sample 5 (d = 0.4738, Class 1). Because there are two 1s and one 0, the majority vote predicts **Class 1 (Diabetic)** for this test instance.

**Part 4: Model Evaluation & Bonus**

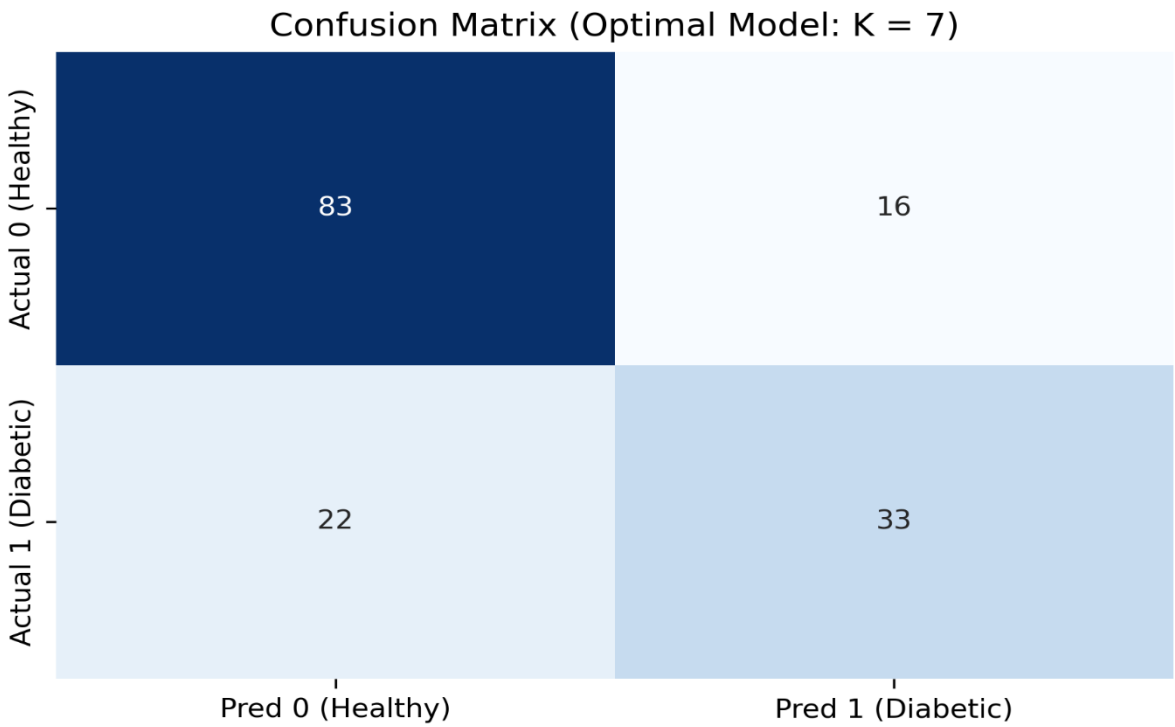
The algorithm was evaluated across the test set using K=3, K=5, and K=7. To fulfill the bonus requirement, Logistic Regression was also applied to the exact same preprocessed data to serve as a baseline comparison.

**Model Performance Comparison Table:**

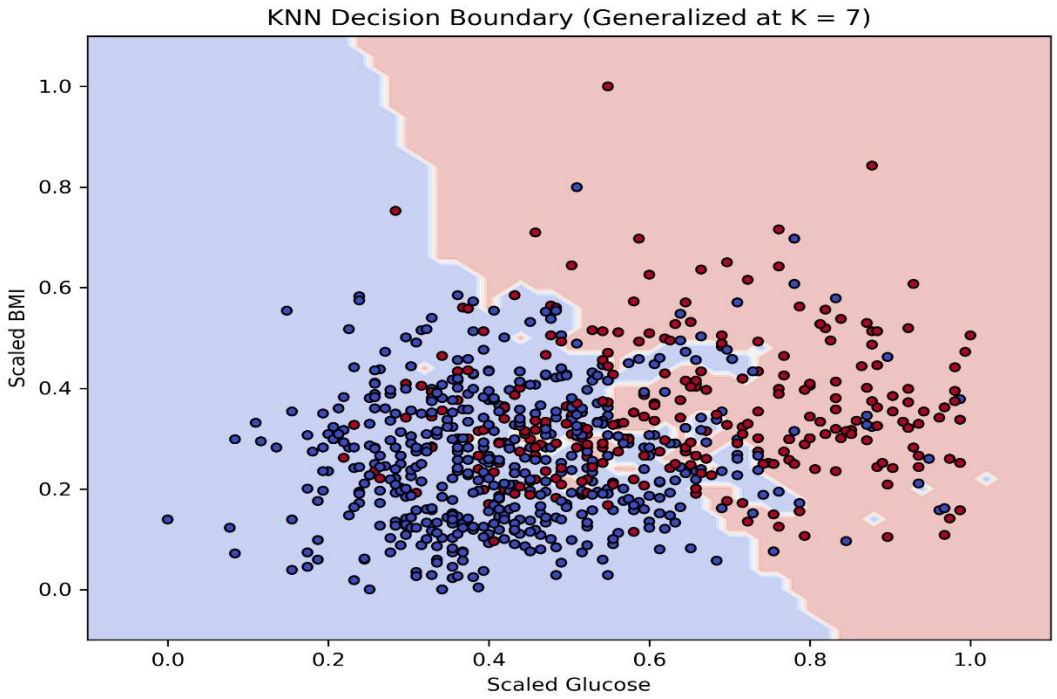
| Model Type | Overall Accuracy | True Positives | False Positives | True Negatives | False Negatives |
|---|---|---|---|---|---|
| KNN (K = 3) | 72.08% | 35 | 23 | 76 | 20 |
| KNN (K = 5) | 74.68% | 36 | 20 | 79 | 19 |
| KNN (K = 7) | 74.68% | 34 | 18 | 81 | 21 |
| Logistic Regression | 76.62% | 35 | 16 | 83 | 20 |




*Part 4: Model Performance Evolution*




*Part 4: Confusion Matrix*




*Part 4: KNN Decision Boundary*

**1. Which value of K performed best?** Based on the accuracy scores, K=5 and K=7 tied for the best performance among the KNN models at 74.68%. However, analyzing the confusion matrix reveals that K=7 is marginally superior for clinical use, as it reduced the number of false positives (healthy patients incorrectly diagnosed as diabetic) down to 18.

**2. Why does performance change with different K values?** Performance shifts because K fundamentally dictates the size of the "neighborhood" that casts a vote on a prediction. Altering K changes the spatial decision boundaries of the model. A lower K creates highly complex, jagged boundaries that attempt to capture every point, while a higher K smooths those boundaries out to capture broader data trends.

3. What happens when K is too small or too large?

- **Too Small (e.g., K=1):** The model severely overfits. It becomes hyper-sensitive to noise and statistical outliers. A single anomalous patient in the training set will force the model to misdiagnose incoming patients that happen to fall near them geometrically.
- **Too Large (e.g., K=150):** The model underfits. The neighborhood becomes so massive that it loses all local context, effectively defaulting to predicting the majority class of the entire dataset regardless of the specific patient's unique metrics.

**Part 5: Analysis and Reflection**

Executing this experiment practically demonstrated both the elegance and the constraints of instance-based machine learning.

One of the primary **strengths** of the K-Nearest Neighbors algorithm is its structural simplicity and high degree of interpretability. Because it is a "lazy learner," it does not make rigid, underlying mathematical assumptions about how the data is distributed (unlike the Logistic Regression model tested in the bonus section). For real-world medical diagnostics, this interpretability is immensely valuable. If a physician asks *why* the model diagnosed a patient as diabetic, the algorithm can be queried to literally point to the specific historical patient records (the nearest neighbors) that share nearly identical biological metrics. It operates on logical precedent rather than abstract probability weights, making it a highly transparent tool.

However, the experiment also highlighted brutal **limitations**. My key observation was that KNN is incredibly sensitive to the scale of the data. As visualized in the boxplot comparisons, without Min-Max normalization, the algorithm would have been functionally useless. Unscaled, a feature with high numerical values mathematically erases the impact of smaller-scale, yet vital, features like the Diabetes Pedigree Function. Furthermore, the computation cost at the prediction phase is a significant drawback. Because KNN does not derive an equation during a "training" phase, it must compute the Euclidean distance to every single training instance every time it makes a new prediction.

In terms of **appropriateness**, KNN is highly suitable for smaller, properly standardized datasets where explainability is a strict priority. However, it is largely inappropriate for massive, high-dimensional datasets or real-time diagnostic systems requiring instantaneous outputs, where an algorithm like Logistic Regression (which outperformed KNN in our test with 76.62% accuracy) would be far more efficient and scalable.